Project Infinity

Codebase Scaling & UX Flow Checklist

Work through these phases in order. Each phase is self-contained — complete one before starting the next to avoid breaking the running app.

1 Lock in constants.js

No logic change — pure extraction. Do this first, it unblocks everything.

Why

XP values, level thresholds, streak rules, and badge triggers are likely scattered across curriculum.js, state/, and views/. Centralising them means you tune game balance in one file forever.

- [ ] **Create src/data/constants.js** `file`
  Export all numeric game values as named constants

- [ ] **Move XP values here** `extract`
  e.g. XP_PER_LESSON, XP_PER_REVIEW, XP_BOSS_BATTLE

- [ ] **Move level thresholds here** `extract`
  e.g. LEVEL_THRESHOLDS = [0, 100, 250, 500 ...]

- [ ] **Move badge trigger conditions here** `extract`
  e.g. BADGE_RULES = [{ id, condition }]

- [ ] **Move streak config here** `extract`
  e.g. STREAK_GRACE_HOURS = 28

- [ ] **Update all import sites** `refactor`
  Find & replace hardcoded values with constant names

- [ ] **Smoke test — run app, confirm XP + levels unchanged** `test`

---

2 Wrap persistence in storageAdapter.js

Makes the backend swap (build step #2) a one-file change, not a project-wide refactor.

Why

If src/state/ writes directly to localStorage, every future backend integration requires hunting down storage calls across the codebase. A StorageAdapter gives you one interface: get(), set(), clear(). Swap the implementation once when you add a backend.

- [ ] **Create src/state/storageAdapter.js** `file`

- [ ] **Define interface: get(key), set(key, val), clear()** `interface`

- [ ] **Implement with localStorage under the interface** implement
  Behaviour identical to current — just wrapped

- [ ] **Replace all direct localStorage calls in state/** refactor
  Import and use storageAdapter everywhere

- [ ] **Add a getAll() method for future export feature** extend
  Returns full state snapshot — needed for build step #2

- [ ] **Smoke test — progress, streaks, XP persist across reload** test

---

3 Extract badgeEngine.js

Most-touched logic as content grows. Must be independently testable.

Why

Badge logic embedded in state or views means every new badge requires digging through unrelated code. A pure function checkBadges(state) → badges[] is easy to test, easy to extend, and safe to call from anywhere.

- [ ] **Create src/services/badgeEngine.js** file

- [ ] **Define checkBadges(state) → badges[] as pure function** implement
  No side effects — takes state, returns earned badges

- [ ] **Move all badge trigger logic here from state/views** extract
  Use BADGE_RULES from constants.js

- [ ] **Wire badgeEngine into state update cycle** wire
  Call after every XP/streak/lesson state change

- [ ] **Write unit tests for at least 3 badge conditions** test
  e.g. first lesson, 7-day streak, boss battle win

- [ ] **Test boss battle unlock condition separately** test
  Boss battles are the highest-stakes trigger

- [ ] **Smoke test — earn a badge manually, confirm display** test

---

4 Standardise view interfaces

Do one view at a time. Each view exports init(), render(state), destroy() — nothing else.

Why

Views that reach into each other or call main.js directly create hidden coupling. Standardised interfaces mean new views slot in without touching existing code.

Contract for every view file:

```
init() — one-time setup (event listeners, DOM refs). render(state) — pure re-render from state. destroy() — clean up listeners before route change.
```

- [ ] **Refactor Dashboard view first** `view`
  Lowest risk — most isolated

- [ ] **Refactor Lesson view** `view`

- [ ] **Refactor SQL Practice view** `view`
  Wraps sqlEngine calls — use QueryService (Phase 5 prep)

- [ ] **Refactor Badges/Boss Battle view** `view`

- [ ] **Ensure no view imports another view directly** `rule`
  Views communicate via state only, never directly

- [ ] **Remove view-specific logic from main.js** `refactor`
  main.js should only bootstrap and hand off

- [ ] **Smoke test each view after refactoring it** `test`

---

5 Add router + event bus

Add last — once views are clean, the router slots in with zero breakage.

Why

A hash-based router (#/dashboard, #/practice, #/review) decouples navigation from main.js. An event bus (~20 lines, no library) lets modules signal each other without direct imports, which is the key to adding new features without touching old files.

- [ ] **Create src/router.js with hash-based routing** `file`
  register(path, view), navigate(path), init()

- [ ] **Register all 4 views as named routes** `wire`
  #/dashboard, #/practice, #/review, #/badges

- [ ] **Move view-switching logic from main.js to router** `refactor`
  main.js becomes: import router → router.init()

- [ ] **Create src/utils/events.js — simple EventEmitter** `file`
  on(event, fn), emit(event, data), off(event, fn)

- [ ] **Replace direct cross-module function calls with events** `refactor`
  e.g. emit("xp:updated", newXP) instead of calling view directly

- [ ] **Test deep-linking — open #/practice directly in browser** `test`

- [ ] **Test back/forward navigation works correctly** `test`

---

6 Scale new features in

After Phases 1–5, every new feature is additive. No refactoring required.

With the foundation set, these build steps from the project plan each map cleanly to one layer:

Feature	Layer to extend	Unblocked by
Spaced repetition cards	New view + badgeEngine	Phase 4 (view interface)
Rich company case packs	src/data/curriculum.js	Phase 1 (constants)
Lesson history export	storageAdapter.getAll()	Phase 2 (storage)
Backend / user accounts	storageAdapter swap	Phase 2 (storage)
Instructor dashboard	New view via router	Phase 5 (router)
AI Mentor (live)	api/mentor.js + new route	Phase 5 (router)
Unit test suite	badgeEngine + queryService	Phase 3 (badge engine)

Rule: every new feature gets a new file. Never extend main.js, never let views import views, never hardcode values that belong in constants.js.